

03 - Jugutier - Empresa mensajería

Universidad del Quindío
Programa de Ingeniería de Sistemas y Computación
Programación III - Parcial 1
Docente: Julián E. Gutiérrez Posada

Para desarrollar el proyecto pueden usar internet, la documentación oficial, los asistentes del curso, cualquier otra herramienta de inteligencia artificial y la ayuda de otras personas. Usar IA no baja la nota. Lo que se evalúa es que cada uso quede registrado y justificado, y que cualquier integrante pueda explicar y modificar cualquier línea del código en la sustentación.

La única restricción tiene que ver con el alcance. El código debe resolverse con los temas vistos hasta la guía de colecciones, por lo que no se acepta:

- recursividad (funciones que se llaman a sí mismas);
- `destructure` ni structs propios;
- lectura o escritura de archivos con `File`;
- procesos (`spawn`, `Task`, `Agent`, `GenServer`);
- proyectos creados con `mix` ni librerías externas;
- `try/rescue` para validar datos. Los errores se manejan con tuplas `{:ok, valor}` y `{:error, motivo}`.

Los recorridos se hacen con `Enum` o con comprehensions (`for`). Las herramientas de IA suelen proponer recursividad o structs aunque se les pida lo contrario, así que detectar esas respuestas y adaptarlas también hace parte del trabajo.

Parcial 1. Liquidación semanal de una empresa de mensajería

El problema

Una empresa local de mensajería contrata repartidores independientes para realizar entregas en diferentes zonas de la ciudad. Cada vez que un repartidor finaliza un servicio, el sistema registra la zona de destino, la distancia recorrida y el retraso de la entrega respecto del tiempo estimado.

La empresa paga cada servicio de acuerdo con la distancia recorrida. Sin embargo, el valor puede aumentar o disminuir dependiendo de la puntualidad. Además, los repartidores que recorren una determinada distancia total durante el día reciben una bonificación por productividad.

Algunos repartidores alquilan una bicicleta eléctrica de la empresa y deben pagar una tarifa por cada día en el que hayan realizado al menos un servicio válido.

Al finalizar la semana, la empresa debe calcular cuánto debe pagar a cada repartidor y generar diferentes reportes de operación.

Parámetros

Parámetro	Valor
Tarifa base por kilómetro	\$2.500
Meta diaria de la empresa	500 km
Días de operación	6, numerados del 1 al 6
Máxima distancia de un servicio	45 km
Kilómetros diarios para bonificación	80 km
Bonificación diaria	\$15.000
Alquiler de bicicleta	\$10.000 por día trabajado

Estos valores deben definirse como atributos del módulo correspondiente.

Formato de los datos

Los datos estarán en el módulo `Datos`, almacenado en `datos.exs`.

```
defmodule Datos do
  def repartidores do
    [
      %{codigo: "M01", nombre: "Ana Torres", bicicleta: true},
      %{codigo: "M02", nombre: "David López", bicicleta: false}
      # ...
    ]
  end

  def zonas do
    [
      %{id: "Z1", nombre: "Centro", area: 6.5},
      %{id: "Z2", nombre: "Norte", area: 10.2}
      # ...
    ]
  end

  def servicios do
    [
      %{repartidor: "M01", zona: "Z1", dia: 1, kilometros: 18, retraso: 3},
      %{repartidor: "M01", zona: "Z2", dia: 1, kilometros: 25, retraso: 14}
      # ...
    ]
  end
end
```

Sobre los datos:

- `bicicleta` indica si el repartidor utiliza una bicicleta eléctrica de la empresa.
- `kilometros` puede ser entero o decimal.
- `retraso` corresponde a los minutos transcurridos entre la hora estimada y la hora real de entrega.
- Un valor negativo de `retraso` significa que la entrega se realizó antes de la hora prevista.
- `area` representa el área aproximada de la zona, expresada en km².

Los registros se recopilan durante el día desde distintos dispositivos y posteriormente se consolidan sin revisión manual. Por esta razón pueden existir errores y el programa debe validarlos antes de utilizarlos.

Los datos llegan inicialmente en listas.

El conjunto de datos del grupo debe incluir:

- mínimo 10 repartidores;
- mínimo 4 que utilicen bicicleta;
- 4 zonas;
- servicios durante los 6 días;
- mínimo 80 servicios válidos;
- al menos dos servicios inválidos por cada motivo de rechazo.

Reglas de negocio

1. Validación de servicios

Los servicios se revisan exactamente en el siguiente orden:

Orden	Regla	Motivo
1	El repartidor existe	:repartidor_desconocido
2	La zona existe	:zona_desconocida
3	El día es un entero entre 1 y 6	:dia_invalido
4	Los kilómetros son un número mayor que 0 y máximo 45	:kilometros_fuera_de_rango
5	El retraso es numérico y se encuentra entre -30 y 180 minutos	:retraso_invalido

Una función debe devolver:

```
{:ok, servicio}
```

o

```
{:error, motivo}
```

Si un servicio incumple más de una condición solo se registra el primer error de acuerdo con el orden anterior.

Los servicios rechazados no participan en ningún cálculo posterior, excepto en R1.

La función principal de validación debe encadenar las verificaciones mediante `with`.

2. Valor de un servicio

El valor inicial de cada servicio válido es:

$\text{kilómetros} \times \text{tarifa base}$

Posteriormente se aplica un ajuste en función de la puntualidad:

Retraso	Ajuste
Hasta 0 minutos	Bonificación del 8 %
Más de 0 y hasta 10 minutos	Sin ajuste
Más de 10 y hasta 30 minutos	Descuento del 10 %
Más de 30 minutos	Descuento del 25 %

Por ejemplo, una entrega realizada 5 minutos antes de la hora prevista tiene un **retraso** de **-5** y recibe la bonificación del 8 %.

3. Bonificación por productividad

Si un repartidor acumula 80 km o más durante un mismo día, considerando únicamente servicios válidos, recibe una bonificación de \$15.000.

La distancia se suma por repartidor y por día.

4. Alquiler de bicicleta

Un repartidor que utiliza bicicleta eléctrica paga \$10.000 por cada día trabajado.

Se considera día trabajado aquel en el que tenga al menos un servicio válido.

5. Liquidación

Para cada repartidor:

```
neto =  
suma del valor de sus servicios  
+ bonificaciones  
- alquiler de bicicleta
```

Todos los repartidores deben aparecer en la liquidación, incluso aquellos que no tengan servicios válidos.

En ese caso, sus valores deben ser cero.

Reportes

El programa imprime los siguientes reportes en este orden.

#	Reporte
R1	Servicios rechazados, motivo de cada rechazo y cantidad de rechazos por motivo.
R2	Kilómetros recorridos en cada zona y densidad de recorrido, calculada como kilómetros / área, ordenados de mayor a menor densidad. Una zona sin servicios válidos aparece con 0 km.
R3	Kilómetros recorridos por toda la empresa en cada uno de los 6 días e indicación de si se alcanzó la meta de 500 km. Al final debe indicarse si se alcanzó todos los días y si se alcanzó al menos un día.
R4	Liquidación de todos los repartidores, numerada y ordenada por neto de mayor a menor. Debe incluir kilómetros, valor de servicios, bonificaciones, alquiler y neto.
R5	Repartidor que recorrió más kilómetros cada día. En caso de empate aparecen todos los empatados. Al final se muestra quién ocupó el primer lugar en más días.
R6	Repartidor con mejor puntualidad entre quienes tengan por lo menos 3 servicios válidos.
R7	Total pagado a todos los repartidores durante la semana y costo promedio pagado por kilómetro.
R8	Repartidores que realizaron al menos un servicio válido en todas las zonas.

Puntualidad ponderada de R6

Para comparar a los repartidores en R6 se utiliza el retraso promedio ponderado por kilómetros:

```
suma(retraso × kilometros) / suma(kilometros)
```

El mejor resultado corresponde al menor retraso ponderado.

Los valores negativos son posibles y representan una entrega anticipada. Por tanto, entre **-4** y **3**, el mejor resultado es **-4**.

En el documento deberán:

1. explicar por qué este valor puede ser diferente del promedio simple de los retrasos;
 2. mostrar un ejemplo tomado de sus propios datos donde ambos promedios sean diferentes;
 3. explicar por qué tiene sentido que un servicio de mayor distancia tenga más peso en esta medida.
-

Interacción con el usuario

Servicio adicional

Antes de generar los reportes, el programa solicita un único servicio adicional:

```
Ingrese un servicio adicional  
(repartidor;zona;dia;kilometros;retraso)  
o Enter para omitir:
```

Ejemplo:

```
M03;Z2;4;22.5;-3
```

La entrada se rechaza con

```
{:error, :formato_invalido}
```

cuando:

- no contiene exactamente cinco campos;

- el día no puede convertirse a entero;
- los kilómetros no son numéricos;
- el retraso no es numérico.

Si el formato es válido, se aplican posteriormente las cinco reglas de validación.

Un servicio válido se incorpora a todos los reportes.

El programa debe informar al usuario si el servicio:

- fue agregado;
- fue rechazado por formato;
- fue rechazado por alguna regla de validación;
- o no se ingresó porque se presionó Enter.

Comprobante del repartidor

Al terminar los reportes, el programa solicita el código de un repartidor.

Para un código válido muestra un comprobante de pago con:

- nombre y código;
- detalle de los días trabajados;
- kilómetros de cada día;
- valor de los servicios de cada día;
- bonificación de cada día;
- suma del valor de servicios;
- suma de bonificaciones;
- descuento por alquiler de bicicleta;
- neto a pagar.

Solo deben aparecer los días en los que exista por lo menos un servicio válido.

Si el código no corresponde a ningún repartidor, el programa debe informar la situación y continuar normalmente.

Investigación: combinación de operaciones de dos empresas

Para el ejercicio de `Map.merge/3`, una empresa aliada informa los kilómetros recorridos por sus repartidores durante la semana mediante:

```
empresa_aliada = %{  
  1 => 580.5,  
  2 => 430,  
  3 => 510,  
  5 => 625,  
  7 => 180  
}
```

Escriban una función que combine este mapa con el mapa de kilómetros diarios producido en R3.

Cuando un día aparezca en ambos mapas deben sumarse los kilómetros.

La combinación debe realizarse utilizando:

```
Map.merge/3
```

En el documento expliquen:

1. qué resultado produciría `Map.merge/2`;
2. por qué dicho comportamiento no resuelve el problema planteado;
3. qué sucede con el día 7 de la empresa aliada.

Las demás exigencias del examen se mantienen:

- diseño y justificación de colecciones;
- identificación de funciones puras e impuras;
- elección y justificación de estructuras de control;
- separación modular;
- función `ranking/2` configurada mediante keyword lists;
- mediciones con `:timer.tc/1`;
- bitácora y reflexión sobre uso de IA;
- sustentación y modificación del programa en vivo.

Se mantienen también las restricciones generales del curso: no utilizar recursividad, `defstruct`, lectura o escritura de archivos, procesos, proyectos `mix`, librerías externas ni `try/rescue` como mecanismo de validación.

Entrega

1. Un repositorio en GitHub, que contenga:
 - los archivos `.exs` del programa, cada uno con los nombres de los integrantes en un comentario al inicio
 - el `datos.exs` del grupo
 - El historial del repositorio se usa para confirmar la participación de cada integrante, así que deben verse commits de todos a lo largo del desarrollo.
- Un documento PDF, con el diseño (Parte A), la explicación de R6, la investigación (Parte C), la salida completa del programa con sus datos y la Parte D.

Criterios de evaluación

Parte	Criterio	Valor
A	Diseño argumentado: elección de colecciones y estructuras de control, separación entre funciones puras e impuras	0.6
B	Validación correcta, con el orden y los motivos exigidos	0.7
B	Liquidación correcta según las reglas y el ejemplo	0.8
B	Reportes completos y correctos con datos de prueba	1.0
B	Interacción con el usuario sin fallos ante entradas inválidas	0.5
C	Investigación: keyword lists (0.3), Map.merge/3 (0.2) y mediciones propias con su explicación (0.3)	0.8
D	Bitácora, caso de error y reflexión honestos y justificados	0.6
	Nota del proyecto	5.0

La nota final de cada integrante se calcula así:

`nota final = nota del proyecto × factor de sustentación`

Por ejemplo, si el proyecto obtiene 4.0 y un integrante recibe un factor de 0.8, su nota final es 3.2. Otro integrante del mismo grupo con factor 1.0 obtiene 4.0.

En toda la Parte B también se tienen en cuenta la nomenclatura, la documentación y la organización del código. Si aparecen temas por fuera del alcance (recursividad, structs, archivos o procesos), el punto donde se usen se califica con cero.